Lesson 1 — RTL modelling with SystemVerilog

SYSTEMS VERIFICATION · 2026/2027

Enrico Fraccaroli · CISD, University of Verona

Combinational logic, registered state, and the diagnostics that tell them apart

Learning goals

By the end of this lesson you should be able to:

- distinguish combinational logic from registered state;
- read and write small SystemVerilog RTL modules;
- reason about HDL concurrency rather than software execution order;
- choose blocking and non-blocking assignments deliberately;
- recognize incomplete combinational assignment and unintended storage;
- use parameters and compile-time `for` loops;
- drive a compiler, a simulator and a waveform viewer by hand;
- read a VCD trace as evidence about *when* a value changed.

How this lesson works

Two halves:

1. The first half is **six small files**, each one a single language idea. You compile them, run them, and read what came out. Nothing in them prints a verdict.
2. The second half is **one empty module**. You write it, and a checker judges it.

The six files are not an exercise in reading code. Each one exists to make a compiler or a simulator *say something*, and your job is to notice what.

Everything under `build/` is generated output. Delete it whenever you like.

HDL describes hardware

A hardware description is **not a sequential program** executed from top to bottom.

Independent hardware structures operate concurrently:

```
combinational logic --> register --> combinational logic
                            ^
                            |
                          clock
```

Only the register is clocked. The combinational parts have no clock: they simply follow their inputs.

Source order helps us organize the description. It does not serialize the hardware.

Simulation and synthesis answer different questions

This comes first because it is why half of today matters.

Simulation asks:

> *What behaviour occurred over time, for these stimuli?*

Synthesis asks:

> *What hardware structure did this description ask for?*

Right in simulation, wrong in hardware

A description can simulate exactly as you intended and **still ask for hardware you never wanted**. Nothing in the simulation tells you so, because the simulator was never asked that question.

The gap, and what reports it

That gap is what a compiler warning is for. Three of them appear today, and each one reports a circuit you did not mean to describe.

You will not run a synthesizer in this course. Turning a description into a netlist of gates and flip-flops, and then testing that netlist, is the second part of the course. Here the compiler's opinion about the structure you asked for is the whole of the evidence, which is a reason to read it rather than scroll past it.

Module anatomy

A module defines a hardware boundary:

```
module example (
    input  logic a,
    input  logic b,
    output logic y
);

  logic local_signal;

  // RTL statements

endmodule
```

Ports and local signals

Ports connect the module to its environment. Local signals connect structures inside it.

Signals in this lesson

Every signal you will meet today is declared `logic`, whether it is a port or a local signal:

```
logic        valid;   // one bit
logic [7:0]  data;    // an 8-bit vector
```

That is the whole type vocabulary this lesson needs. The same `logic` declaration covers both combinational signals and registered state; what makes a signal a register is how it is assigned, not how it is declared.

SystemVerilog signals can also carry the values `X` (unknown) and `Z` (high impedance). Lesson 1 does not reason about them.

Combinational logic: procedural form

A procedural combinational block responds to its inputs:

```
always_comb begin
  if (cond)
    y = a | b;
  else
    y = c & d;
end
```

Every path assigns `y`.

That is intended hardware, not formatting.

Combinational logic: continuous form

The same relation can often be written as a continuous assignment:

```
assign y = cond ? (a | b) : (c & d);
```

Both forms can describe the same combinational hardware.

Choose the form that makes the intended relation easiest to read.

Mini-checkpoint 1 — same hardware?

No files. Decide from the two snippets only.

```
always_comb begin
  if (sel)
    y = a;
  else
    y = b;
end
```

```
assign y = sel ? a : b;
```

Mini-checkpoint 1 — the questions

Questions:

1. Do these describe the same Boolean function?
2. Is any state required?
3. What hardware structure would you expect?

Decide before turning the page.

Mini-checkpoint 1 — answer

1. Yes: both describe the same Boolean function.
2. No state is required. Every path assigns `y`, so nothing has to be remembered.
3. One 2-to-1 selection structure, a multiplexer driven by `sel`.

Both forms are legitimate. The choice between them is about readability, not about the hardware you get.

The first specimen puts these two side by side and drives all eight input combinations, so you can check the claim rather than take it.

Compile one yourself

Nothing is hidden behind a target. You type this:

```
mkdir -p build/obj/combinational build/waves

verilator -Wall -f verilator.cfg \
    -Mdir build/obj/combinational --top-module tb_combinational -o sim \
    styles/01_combinational.sv
```

Verilator does not create that directory, which is what the `mkdir` is for.

Three steps, three tools

A compiler, a simulator, a viewer. Every later lesson in this course is these three steps with more in the middle.

```
./build/obj/combinational/sim

vcdtui build/waves/combinational.vcd --scope tb_combinational \
       -s sel,a,b,y_proc,y_cont --dump
```

The simulator writes the trace; the viewer reads it. `--dump` prints one row per tick at which something changed.

Check it yourself, now

Run those three commands on `01_combinational.sv`, and answer the question you just decided on paper: do `y_proc` and `y_cont` ever differ?

This is the first compiler command most of you will have typed. It is worth getting that out of the way now, while there is still an hour to recover from whatever it does.

Why complete assignment matters

Consider:

```
always_comb begin
  if (enable)
    y = d;
end
```

When `enable=0`, what value should `y` have?

If the description means "keep the previous value", some hardware must remember that value.

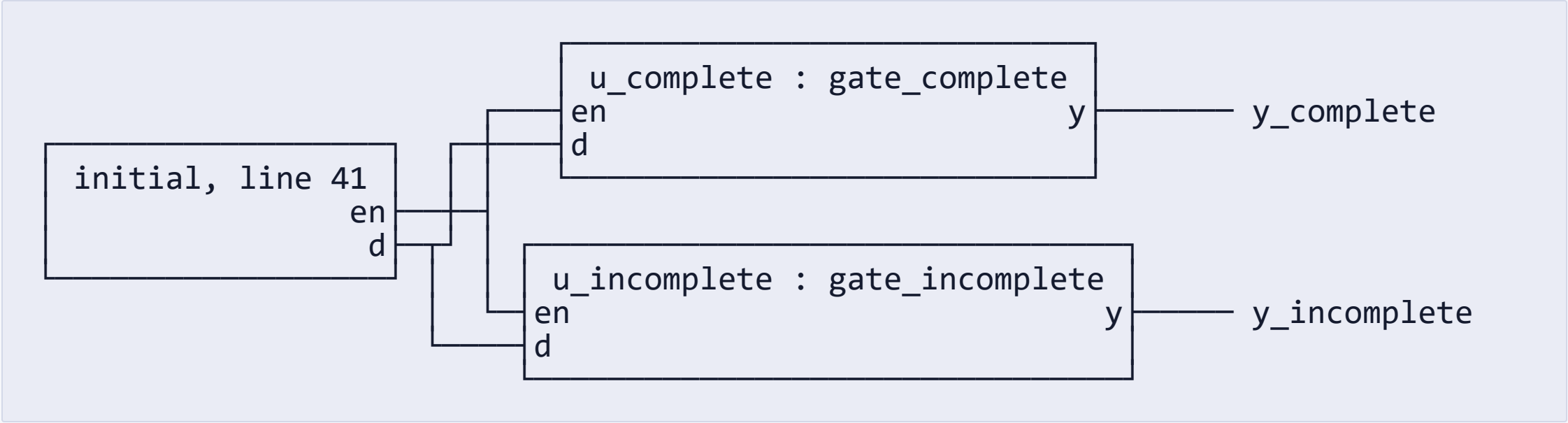
Where the storage comes from

```
missing assignment on a path
          |
          v
  previous value must survive
          |
          v
      storage element
```

What that storage element is

That storage element is a **latch**: state you never asked for, in a block you called combinational.

The second specimen, drawn



One stimulus, two gates, two outputs. They look the same from outside, which is why the trace has to be read.

Mini-checkpoint 2 — combinational or storage?

No files. Classify each fragment from the snippets only.

A:

```
always_comb begin
  y = 1'b0;
  if (enable)
    y = d;
end
```

Mini-checkpoint 2 — fragment B

B:

```
always_comb begin
  if (enable)
    y = d;
end
```

Classify both before turning the page.

Mini-checkpoint 2 — answer

A is combinational. The default assignment `y = 1'b0` means every path through the block defines `y`.

B requires storage. When `enable=0` nothing assigns `y`, so the description asks for the previous value to survive. Something must remember it.

Mini-checkpoint 2 — the compiler agrees

The compiler reaches the same conclusion, and says so:

```
%Warning-LATCH: Latch inferred for signal ...
                (not all control paths of combinational always assign a value)
```

The second specimen holds both versions. Watch for the moment in the trace where they disagree, and for the moment where they agree **by luck rather than by logic**.

Registered state

A flip-flop changes its stored output on a clock edge:

```
always_ff @(posedge clk) begin
  q <= d;
end
```

Between active clock edges, `q` retains its registered value.

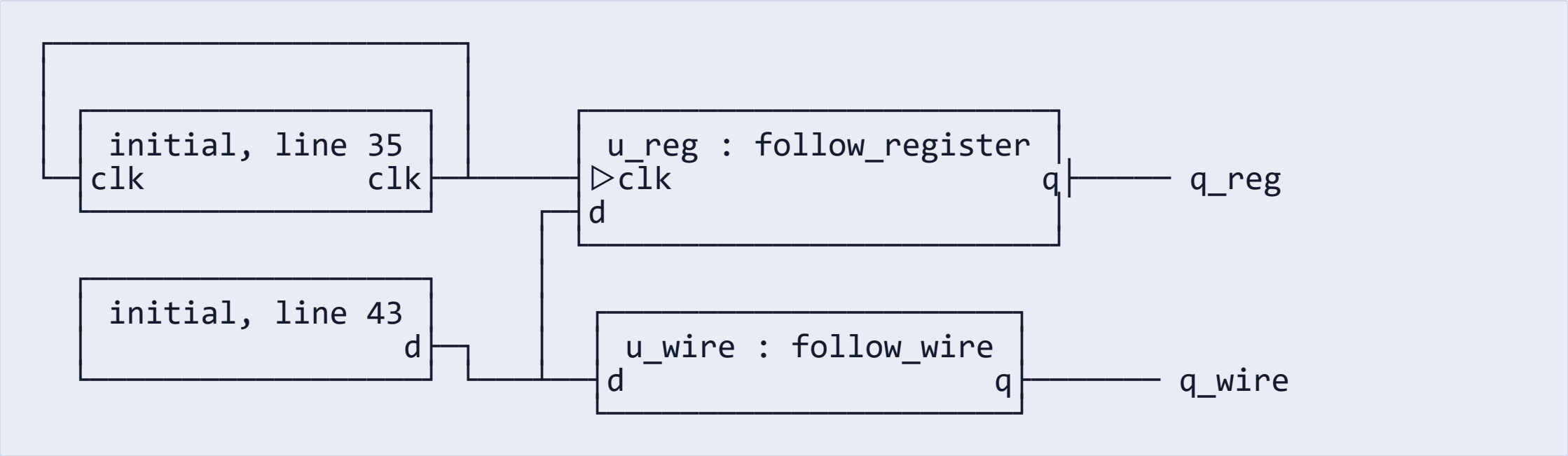
The question to ask is:

> ***What state element does this description represent, and when may that state change?***

A wire and a register, same input

A wire and a register driven by the same input are the cleanest way to see it. One follows its input continuously; the other samples it, and only at an edge. An input pulse that lives and dies between two edges **never reaches the register at all**.

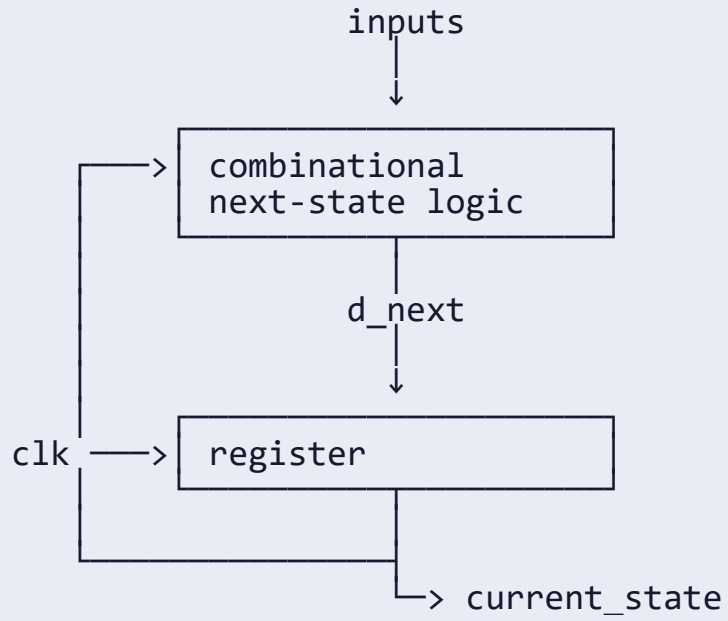
The third specimen, drawn



`follow_register` carries the clock triangle and `follow_wire` does not.

Combinational logic feeding a register

A typical RTL path combines both kinds of logic:



What that shape is for

The combinational part may react immediately to its inputs. The registered output changes only on the active clock edge.

This shape is the thing you will build in the second half of the lesson.

Blocking versus non-blocking assignment

Use blocking assignment `=` for ordinary combinational procedural logic:

```
stage1 = d;
q      = stage1;
```

Non-blocking assignment, for state

Use non-blocking assignment `<=` for clocked state:

```
stage1 <= d;
q      <= stage1;
```

For the non-blocking form, right-hand sides are evaluated from the pre-update state of that clock event; the registered updates take effect together afterward.

```
combinational logic -> =
clocked state        -> <=
```

Mini-checkpoint 3 — predict the update

No files. Assume initially `a=1` and `b=2`.

Case A, blocking assignment in a clocked block:

```
always @(posedge clk) begin
  a = b;
  b = a;
end
```

Mini-checkpoint 3 — case B

Case B, non-blocking assignment in a clocked block:

```
always_ff @(posedge clk) begin
  a <= b;
  b <= a;
end
```

After one rising edge, what are `(a,b)`?

Predict both before turning the page.

Mini-checkpoint 3 — answer

Case A gives `(2,2)`. The blocking assignments take effect in order, so the second statement already sees the updated `a`. The original value of `a` is lost and the intended swap does not happen.

Case B gives `(2,1)`. Both right-hand sides are evaluated from the pre-edge state, and the two registered updates take effect together afterwards.

Two spellings, two circuits

Be clear about what kind of difference this is. `=` against `<=` in a clocked block changes **what circuit you described**, not how an optimiser lays one out. Two spellings, two different circuits.

The compiler names it:

```
%Warning-BLKSEQ: Blocking assignment '=' in sequential logic process
```

The fourth specimen runs both and shows you the two circuits in one trace.

One signal, one process

Two procedural blocks writing the same signal is a description with no answer in it:

```
always @(posedge clk) q <= d;
always @(posedge clk) q <= ~d;
```

Nothing here says which one wins. A simulator will still show you a value, and that value is a fact about the simulator, not about the design.

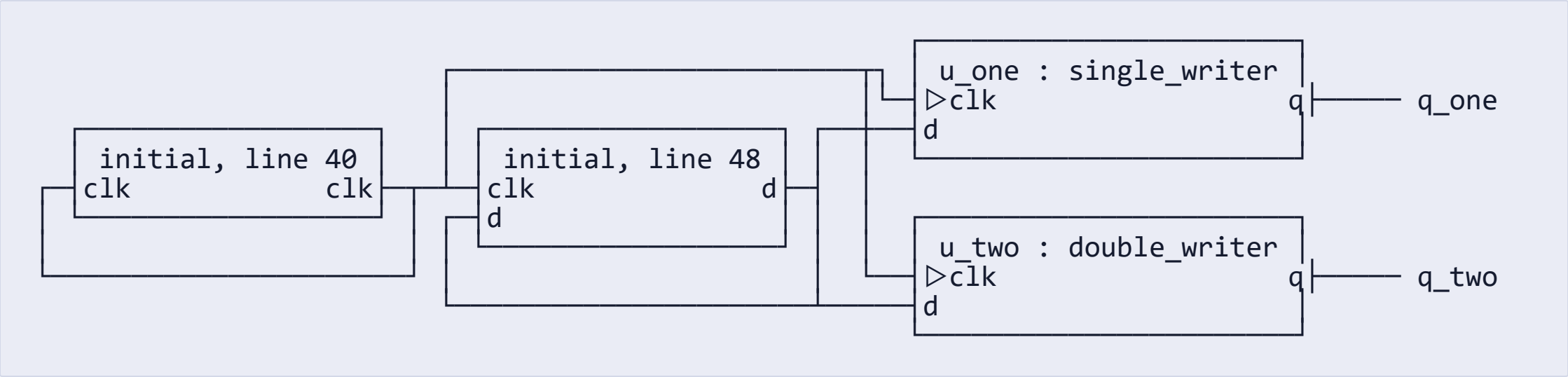
```
%Warning-MULTIDRIVENPROC: Variable written to in always block
                          also written by another always block
```

Why the keyword matters here

Notice those are plain `always` blocks, not `always_ff`. Written as `always_ff` the mistake would be caught for a second, independent reason: `always_ff` is a *promise* about where a signal is written, and a tool can hold you to a promise you made explicitly.

Say what you mean with the specific keyword, because the specific keyword is what lets a tool check you.

The fifth specimen, drawn



`u_two` takes `d` from two places. Nothing in the picture says which one wins, because nothing in the description does either.

Parameters and instantiation shape hardware

A parameter is declared in the module header, with a default:

```
module counter #(
    parameter int WIDTH = 4
) (
    input  logic [WIDTH-1:0] in,
    output logic [WIDTH-1:0] value,
    // ...
);
```

`WIDTH` is already usable in the port list underneath it. The parameter is fixed before any hardware is built, so it can decide how wide that hardware is.

Instantiating it at two widths

The same module is then instantiated at different widths:

```
counter #(.WIDTH(8)) dut8 (
    .clk   (clk),
    .start (start),
    .in    (in8),
    .value (value8)
);
```

The names on the left are ports of the instantiated module. The signals on the right belong to the enclosing scope.

One source file, two different circuits.

Compile-time `for` loops

A synthesizable RTL loop describes replicated hardware when its bounds are known during elaboration:

```
always_comb begin
  for (int i = 0; i < WIDTH; i++) begin
    q[i] = d[WIDTH-1-i];
  end
end
```

When did that loop run?

The loop is **inside** `always_comb`, and that is the point. It is not a loop that runs; it is a description of `WIDTH` wires that all exist at once. The elaborator unrolls it because `WIDTH` is fixed before any hardware is built.

Every bit of `q` is assigned, so the block is complete and no storage appears.

Ask yourself when that loop ran. The honest answer is that it never did: it was unrolled before the simulation started.

The six specimens

Six files, one idea each. The design comes first in every file, the testbench underneath it.

FILE	THE IDEA
`styles/01_combinational.sv`	two spellings of one multiplexer
`styles/02_incomplete.sv`	a combinational path that assigns nothing
`styles/03_registered.sv`	a wire and a register, same input
`styles/04_blocking.sv`	`=` and `<=` in a clocked block
`styles/05_two_drivers.sv`	one signal, two writers
`styles/06_parameters.sv`	one module, two widths

The specimens print no verdict

None of them prints a verdict. Each testbench drives its inputs, records a trace, and stops. The evidence is what the compiler said and what the trace shows. You supply the conclusion.

What those arguments do

A flag you cannot explain is a flag you cannot trust, so these are the ones that decide something:

ARGUMENT	MEANING
`-Wall`	report the style warnings, not only the errors
`-f verilator.cfg`	the flags every build here shares, given to you
`-Mdir`	where generated sources and objects go
`--top-module`	which module is the top of the compiled hierarchy
`-o sim`	call the produced executable `sim`
the last argument	the source file

The two that decide something

Two of the three diagnostics today are reported **only** under `-Wall`. Drop it and they vanish without a word, leaving a clean-looking build of a design that is wrong.

`-Wall` goes **before** `verilator.cfg`, because the file's own settings come after it and would otherwise be undone.

The design and the testbench are compiled together. To the tool, a testbench is simply more SystemVerilog.

Reading a trace is the point, not a chore

A VCD trace answers one question that no amount of reading the source can:

> *When did this value change?*

The code says what the next value should be. Only the trace says at which edge the register took it, and whether anything happened in between.

One viewer argument worth knowing

One argument in the viewer command is worth understanding:

```
--scope tb_combinational
```

It roots the view at the testbench. Without it, a name like `clk` can match several signals: the testbench has one and hands it to every instance.

`--dump` prints one row per change. Leave it out and you get an interactive viewer instead.

The three intentional warnings

Across the six specimens, exactly three kinds of diagnostic appear, and every one of them is deliberate:

```
%Warning-LATCH             x1    the incomplete combinational block
%Warning-BLKSEQ            x2    the clocked blocking assignments
%Warning-MULTIDRIVENPROC   x1    the signal with two writers
```

Whose warning is it?

A warning naming any other file is yours. The one exception is the counter you have not written yet: an empty module reports its outputs as undriven, and that warning disappears the moment the body exists.

None of these stops the build. That is a choice recorded in `verilator.cfg`: a warning that aborted the compile would leave you no trace to look at. Take that setting out and the same specimen refuses to build.

Now the other five

```
make style2   # ... through make style6
```

Each prints every command before running it. Run at least one by hand instead: only one name changes, and it changes in all three commands.

Three of the five warn. Read the warning before running anything.

Read the warning before you run anything. It arrives before the simulation, before the waveform, and it is about the circuit you described rather than about one set of stimuli.

The counter you will write

`rtl/counter.sv` has a module header and nothing else. The body is yours.

There is no reset. At every rising edge of `clk`:

- `start=1` loads `in` (`REQ-CNT-002`);
- otherwise `dec=1` decrements the registered value (`REQ-CNT-003`);
- decrementing stops at zero (`REQ-CNT-006`);
- otherwise the value is held (`REQ-CNT-004`).

The counter: the combinational half

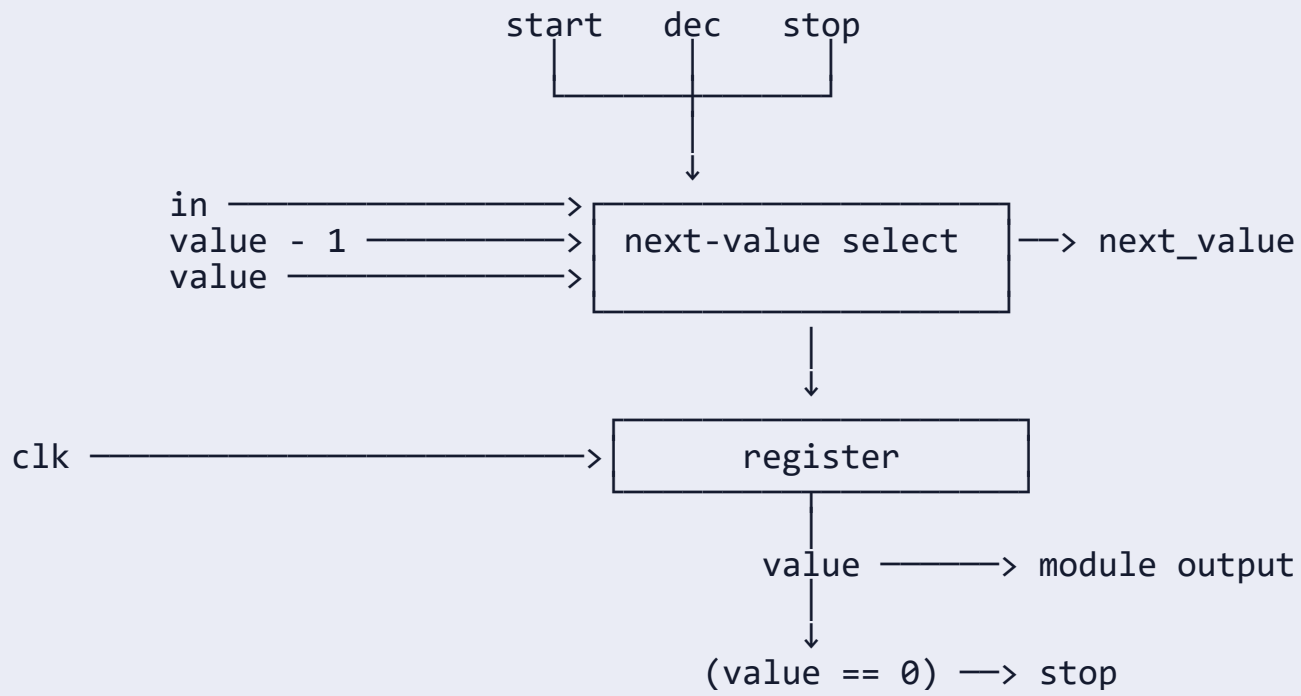
And at all times:

- `stop=1` exactly when the value the counter is **holding** is zero, not when the value it is about to take is zero (`REQ-CNT-007`);
- `start` wins over `dec` when both are high (`REQ-CNT-005`).

`WIDTH` is a parameter and the checker builds your file at two widths (`REQ-CNT-001`). Write it once.

Initialization happens through the load path (`REQ-CNT-008`). Start reasoning at the first load, and ignore whatever the register held before it.

The shape of what you are building



The four pieces, and the loop

Four pieces, and you have met all four this morning: a local signal for the next value, a continuous assignment for `stop`, an `always_comb` that chooses, and an `always_ff` that stores.

The arrow from `value` back into the select is why the counter can hold.

Mini-checkpoint 4 — what happens at the next edge?

No files. Use only the behaviour on the specification slide.

Immediately before a rising edge:

```
value = 5
in    = 12
start = 1
dec   = 1
```

What must `value` become, and which clause decides it?

Mini-checkpoint 4 — the second case

What if instead `value=0`, `start=0`, `dec=1`?

Answer both before turning the page.

Mini-checkpoint 4 — answer

First case: `value` becomes `12`. Both controls are high and the load wins (`REQ-CNT-005`).

Second case: `value` stays `0`. Decrementing is disabled once the registered value has reached zero (`REQ-CNT-006`).

Control priority is observable behaviour, not a detail of how the code is written, so it has to be tested explicitly. The supplied checker does exactly that, at both widths.

Write the counter

```
make check
```

`rtl/counter.sv` is yours and the checker is not. It names a behaviour that is wrong, never a line, so read the name and reread the clause it points at.

Start from the load path and ignore what the register held before it.

What the checker can and cannot tell you

The checker names what is wrong, not where:

```
FAIL counter.start_priority.width4
```

That is a behaviour, and it maps to one clause of the specification. It **does not map to a line of your file**. Reading the name and then rereading the clause is faster than guessing.

What a green run does not say

What it cannot tell you is whether the trace means what you think it means. A value that drops by exactly one is not necessarily a decrement: a load can look identical if you only watch the output. The input that distinguishes them is right there in the same trace.

A green run is necessary and not sufficient. It says the stimulus the checker chose produced the values it expected. Nothing more.

Lesson 1 takeaway

```
combinational logic   assign on every path, or you asked for storage
registered state      one clock, one process, non-blocking
the difference        is in what you described, not in how it ran
```

Three habits worth keeping

Three habits worth keeping:

- **Read the compiler before you read the waveform.** It is talking about your circuit; the waveform is talking about one set of stimuli.
- **Use the specific keyword.** `always_comb` and `always_ff` are promises, and a promise stated explicitly is one a tool can hold you to.
- **Ask when, not just what.** The source says what the next value should be. Only the trace says when the state took it.

Next lesson: how a testbench decides, instead of just driving and recording.